

4. ANDROID APP ARCHITECTURE

SAMS (Student Attendance Management System) - Android Kotlin Compose Application

Table of Contents

1. [Project Overview](#)
 2. [Technology Stack](#)
 3. [Project Structure](#)
 4. [Architecture Layers](#)
 5. [Key Features](#)
 6. [Data Models](#)
 7. [Navigation Flow](#)
 8. [Core Components](#)
 9. [State Management](#)
 10. [Security & Permissions](#)
-

Project Overview

Application: SAMS Android App **Framework:** Android 8.0+ (API 26+) **UI Framework:** Jetpack Compose (Modern declarative UI) **Language:** Kotlin **Architecture:** MVVM + Repository Pattern **Build System:** Gradle with Kotlin DSL

Core Features: - User authentication (Student & Teacher roles) - Real-time attendance marking with GPS + Face verification - Schedule management and class tracking - Push notifications via Firebase Cloud Messaging - Attendance history and reports - Face registration via ML Kit

App Sizes: - Version: 1.0.0+ - Minimum SDK: API 26 (Android 8.0) - Target SDK: API 35 (Android 15) - APK Size: ~45-55 MB

Technology Stack

Core Android Libraries

```
// Kotlin & Coroutines
kotlin-stdlib
kotlinx-coroutines-core
kotlinx-coroutines-android

// Jetpack Components
androidx-appcompat
androidx-activity-compose
androidx-navigation-compose
androidx-lifecycle-runtime
androidx-lifecycle-viewmodel
androidx-room
androidx-datastore-preferences

// Base Android compatibility
// Activity + Compose integration
// Navigation with Compose
// Lifecycle management
// ViewModel for state management
// Local database (SQLite)
// Key-value storage
```

```

// Compose UI
androidx-compose-ui
androidx-compose-material3           // Material Design 3
androidx-compose-foundation
androidx-compose-runtime
androidx-compose-animation

// Networking & Serialization
retrofit2                           // REST API client
okhttp3                             // HTTP client
kotlinx-serialization                // JSON serialization
kotlinx-serialization-json

// Firebase
firebase-analytics
firebase-messaging                   // FCM for push notifications
firebase-crashlytics                 // Crash reporting

// ML Kit
com.google.mlkit:face-detection      // Face detection
com.google.mlkit:vision-common

// Location & GPS
com.google.android.gms:play-services-location

// Image Loading & Processing
coil                                 // Image loading library
androidx-graphics                     // Image processing utilities

// Dependency Injection (Hilt)
com.google.dagger:hilt-android
com.google.dagger:hilt-compiler
androidx-hilt-navigation-compose

// Permissions
com.google.accompanist:accompanist-permissions

// Testing
junit
androidx-test-espresso
mockk

```

Project Structure

```

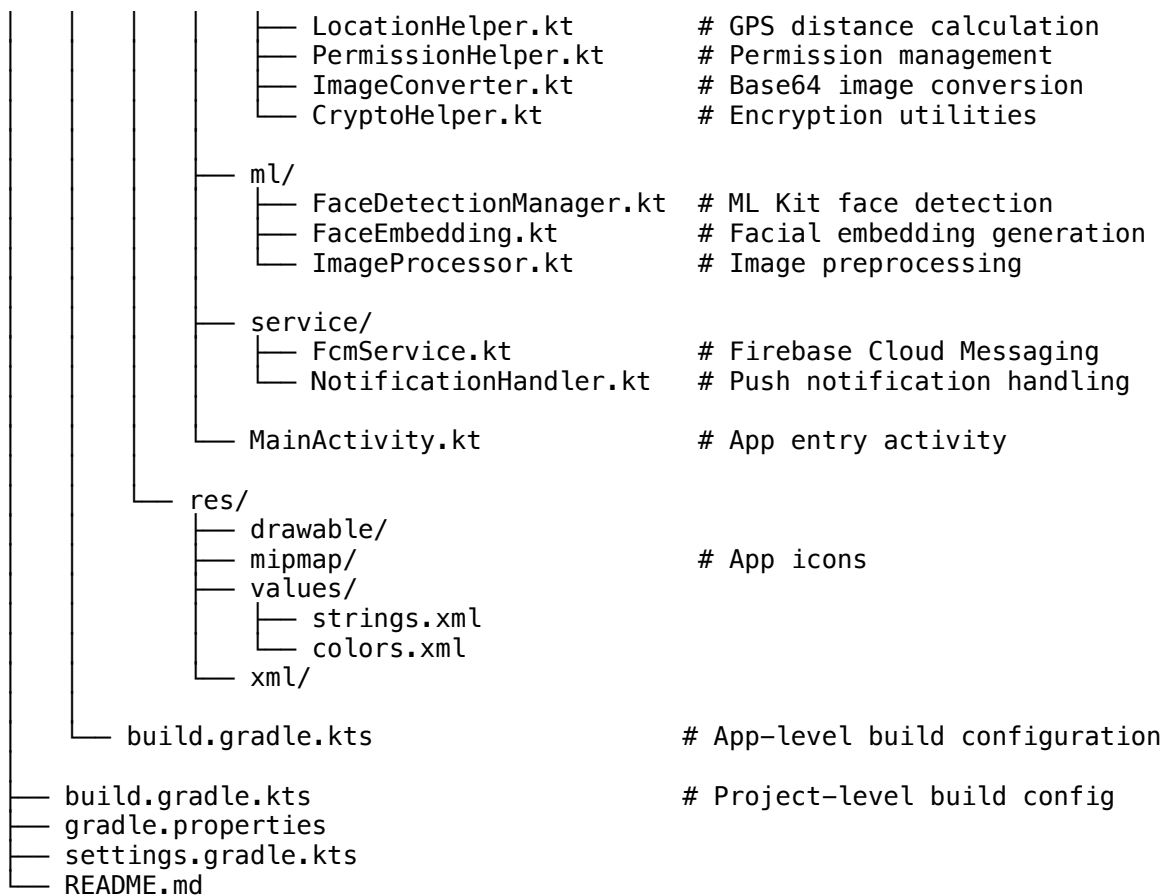
android-kotlin/
├── app/
│   ├── src/main/
│   │   ├── AndroidManifest.xml
│   │   ├── java/com/sams/app/
│   │   │   ├── SamsApplication.kt           # App entry point
│   │   │   └── data/
│   │   │       ├── api/
│   │   │       │   ├── ApiClient.kt        # Retrofit configuration
│   │   │       │   ├── ApiService.kt       # API endpoints definition
│   │   │       │   └── interceptors/        # Network interceptors
│   │   │       ├── models/
│   │   │       │   ├── CommonModels.kt     # Face, Attendance, Notification
│   │   │       │   ├── StudentModels.kt    # Login, Dashboard profiles
│   │   │       │   └── TeacherModels.kt    # Teacher-specific models
│   │   │       └── repository/
│   │   │           └── AuthRepository.kt    # Authentication logic

```

```

├── StudentRepository.kt
├── TeacherRepository.kt
├── AttendanceRepository.kt
├── NotificationRepository.kt
├── local/
│   ├── SessionManager.kt    # User session management
│   ├── PreferenceManager.kt # SharedPreferences wrapper
│   └── UserDao.kt          # Room database
├── di/
│   ├── AppModule.kt        # Hilt dependency injection
│   ├── RepositoryModule.kt
│   └── NetworkModule.kt
├── ui/
│   ├── auth/
│   │   ├── LoginScreen.kt    # Login screen UI
│   │   ├── LoginViewModel.kt # Login state management
│   │   └── LoginUiState.kt   # State definitions
│   ├── student/
│   │   ├── StudentDashboardScreen.kt
│   │   ├── StudentDashboardViewModel.kt
│   │   ├── ScheduleScreen.kt
│   │   ├── AttendanceHistoryScreen.kt
│   │   ├── AttendanceMarkingScreen.kt
│   │   ├── FaceRegistrationScreen.kt
│   │   ├── ProfileScreen.kt
│   │   └── StudentNavGraph.kt
│   ├── teacher/
│   │   ├── TeacherDashboardScreen.kt
│   │   ├── TeacherDashboardViewModel.kt
│   │   ├── StartClassScreen.kt
│   │   ├── ClassAttendanceScreen.kt
│   │   ├── EndClassScreen.kt
│   │   ├── ManualAttendanceScreen.kt
│   │   └── TeacherNavGraph.kt
│   ├── common/
│   │   ├── components/
│   │   │   ├── TopAppBars.kt
│   │   │   ├── BottomNavigation.kt
│   │   │   ├── LoadingDialog.kt
│   │   │   └── ErrorDialog.kt
│   │   ├── cards/
│   │   │   ├── AttendanceCard.kt
│   │   │   ├── ScheduleCard.kt
│   │   │   └── SubjectCard.kt
│   │   └── theme/
│   │       ├── Color.kt      # Color palette
│   │       ├── Typography.kt # Font styles
│   │       └── Theme.kt      # Material3 theme
│   └── navigation/
│       ├── AppNavigation.kt    # Main navigation graph
│       ├── NavDestinations.kt # Navigation routes
│       └── NavigationEvent.kt  # Navigation events
└── util/
    ├── Constants.kt          # Constants & endpoints
    ├── Extensions.kt         # Extension functions
    ├── Logger.kt             # Logging utility
    └── TimeFormatter.kt       # Date/time formatting

```



Architecture Layers

1. Data Layer (Network & Local Storage)

Location: /data/

Handles all data operations (network, database, cache):

Components: - **API Service** (ApiService.kt): Retrofit endpoints - **Repositories** (*Repository.kt): Abstracts data source (API or local) - **Models** (*Models.kt): Kotlin data classes with serialization - **Local Storage** (SessionManager.kt, PreferenceManager.kt)

Example - AuthRepository:

```

class AuthRepository @Inject constructor(
    private val apiService: ApiService,
    private val sessionManager: SessionManager
) {
    suspend fun login(email: String, password: String): Result<LoginResponse> {
        return try {
            val response = apiService.login(LoginRequest(email, password))
            sessionManager.saveUser(response.data.user)
            sessionManager.saveSessionId(response.data.sessionId)
            Result.success(response)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}
  
```

2. Domain Layer (Business Logic)

Location: /ui/ & /util/

Contains use cases and business rules:

Components: - **ViewModels** (*ViewModel.kt): State management - **Repository Interfaces:** Data abstraction - **Domain Models:** Business entities - **Use Cases:** Business logic operations

Example - StudentViewModel:

```
@HiltViewModel
class StudentDashboardViewModel @Inject constructor(
    private val studentRepository: StudentRepository
) : ViewModel() {

    private val _uiState = MutableStateFlow<UiState<StudentDashboard>>(
        UiState.Loading()
    )
    val uiState = _uiState.asStateFlow()

    fun loadDashboard() {
        viewModelScope.launch {
            _uiState.value = UiState.Loading()
            studentRepository.getDashboard()
                .onSuccess { dashboard ->
                    _uiState.value = UiState.Success(dashboard)
                }
                .onFailure { error ->
                    _uiState.value = UiState.Error(error.message ?: "Unknown error")
                }
        }
    }
}
```

3. Presentation Layer (UI)

Location: /ui/

Composable screens and UI components:

Components: - **Screens** (*Screen.kt): Full-screen UI components - **Components** (common/components/): Reusable UI elements - **Theme** (common/theme/): Material Design styling - **Navigation** (navigation/): Screen navigation

Example - StudentDashboardScreen:

```
@Composable
fun StudentDashboardScreen(
    viewModel: StudentDashboardViewModel = hiltViewModel(),
    onNavigateToAttendance: () -> Unit
) {
    val uiState by viewModel.uiState.collectAsState()

    LaunchedEffect(Unit) {
        viewModel.loadDashboard()
    }

    when (val state = uiState) {
        is UiState.Loading -> {
            LoadingDialog()
        }
    }
}
```

```

    }
    is UiState.Success -> {
        DashboardContent(
            dashboard = state.data,
            onAttendanceClick = onNavigateToAttendance
        )
    }
    is UiState.Error -> {
        ErrorDialog(message = state.message)
    }
}
}

```

4. Infrastructure Layer (Utilities & Services)

Location: /util/, /service/, /ml/

Supporting services:

Components: - **Location Services:** GPS handling, distance calculation - **ML Kit Integration:** Face detection & embedding - **FCM Service:** Push notifications - **Permission Management:** Runtime permissions - **Logging & Analytics:** Debugging & monitoring

Key Features

1. Authentication

- Email/password login
- Role-based access (Student/Teacher)
- Session persistence
- Auto-logout on session expiration

Flow:

```

LoginScreen
  ↓ (email, password)
AuthRepository.login()
  ↓ (API call)
Save SessionId & User
  ↓
Navigate to Dashboard

```

2. Attendance Marking (Student)

- Real-time GPS location capture
- Face detection via ML Kit
- Face confidence scoring
- Dual verification (GPS + Face)

Flow:

```

AttendanceMarkingScreen
  ↓ (permission checks)
Request Location → Request Camera
  ↓
Capture Face Image → ML Kit Processing
  ↓ (extract facial embedding)

```

```

Get Teacher Location
  ↓ (calculate distance)
MarkAttendanceRequest:
{
  schedule_id,
  student_lat/long,
  face_confidence,
  face_embedding
}
↓
Backend Verification
├ GPS check (< 50m)
├ Face check (> 75% confidence)
└ Store attendance record

```

3. Class Management (Teacher)

- Start/End class sessions
- Real-time student attendance tracking
- Manual attendance marking
- Session summary

4. Face Registration

- Capture multiple face images
- Generate facial embeddings
- ML Kit face detection
- Encrypted storage on backend

5. Push Notifications

- FCM token registration
- Real-time alerts
- Attendance reminders
- Schedule notifications

Data Models

Core Models

LoginRequest/Response:

```

@Serializable
data class LoginRequest(
    val email: String,
    val password: String,
    val deviceToken: String? = null
)

```

```

@Serializable
data class LoginResponse(
    val success: Boolean,
    val data: LoginData? = null
)

```

```

@Serializable
data class LoginData(

```

```

        val user: LoginUser,
        val sessionId: String,
        val profile: LoginProfile? = null
    )

```

StudentProfile:

```

@Serializable
data class StudentProfile(
    val id: Int,
    val userId: Int,
    val fullName: String,
    val email: String,
    val rollNumber: String,
    val departmentId: Int,
    val departmentName: String,
    val semester: Int,
    val section: String,
    val batchYear: Int,
    val admissionDate: String,
    val faceRegistered: Boolean,
    val profileImage: String? = null
)

```

StudentDashboard:

```

@Serializable
data class StudentDashboard(
    val profile: StudentProfile,
    val overallAttendance: AttendanceStats,
    val subjectWise: List<SubjectAttendance>,
    val recentAttendance: List<RecentAttendance>,
    val lowAttendanceSubjects: List<LowAttendanceSubject>,
    val todaySchedule: List<ScheduleItem>,
    val activeSession: ActiveSession? = null
)

```

Attendance Models:

```

@Serializable
data class MarkAttendanceRequest(
    val scheduleId: Int,
    val latitude: Double,
    val longitude: Double,
    val faceConfidence: Double,
    val faceEmbedding: String? = null
)

```

```

@Serializable
data class MarkAttendanceResponse(
    val success: Boolean,
    val attendanceId: Int? = null,
    val status: String,
    val verificationStatus: String,
    val faceConfidence: Double? = null,
    val distanceMeters: Double? = null
)

```

Navigation Flow

Authentication Flow

```

SplashScreen
  ↓ (check session)
  └─ Session exists → MainActivity
  └─ No session → LoginScreen
      ↓
      LoginScreen → StudentDashboard (or TeacherDashboard)

```

Student Navigation

```

StudentNavGraph:
├─ StudentDashboard
│   ├── ScheduleScreen
│   ├── AttendanceHistoryScreen
│   ├── AttendanceMarkingScreen
│   ├── FaceRegistrationScreen
│   └── ProfileScreen
└─ Menu
    ├── Settings
    └── Logout → LoginScreen

```

Teacher Navigation

```

TeacherNavGraph:
├─ TeacherDashboard
│   ├── ScheduleScreen
│   ├── StartClassScreen → ClassAttendanceScreen
│   │   ├── ManualAttendanceScreen
│   │   └── EndClassScreen
│   └── ProfileScreen
└─ Menu
    ├── Settings
    └── Logout → LoginScreen

```

Core Components

1. State Management with ViewModel

Pattern: MutableStateFlow for reactive state

```

sealed class UiState<out T> {
    data class Loading<T>(val data: T? = null) : UiState<T>()
    data class Success<T>(val data: T) : UiState<T>()
    data class Error<T>(val message: String, val code: Int? = null) : UiState<T>()
    class Idle<T> : UiState<T>()
}

```

```

// Usage in ViewModel
private val _uiState = MutableStateFlow<UiState<StudentDashboard>>(UiState.Idle())
val uiState = _uiState.asStateFlow()

```

2. Dependency Injection with Hilt

```

@Module
@InstallIn(SingletonComponent::class)
object AppModule {

    @Provides
    @Singleton
    fun provideApiService(): ApiService {

```

```

        return Retrofit.Builder()
            .baseUrl("https://api.sams.edu/")
            .addConverterFactory(Json.asConverterFactory("application/json".toMediaType()))
            .client(provideOkHttpClient())
            .build()
            .create(ApiService::class.java)
    }

    @Provides
    @Singleton
    fun provideSessionManager(
        context: Context
    ): SessionManager {
        return SessionManager(context)
    }
}

```

3. Jetpack Compose UI

Material Design 3 - Modern, responsive UI:

```

@Composable
fun StudentDashboardContent(dashboard: StudentDashboard) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        // Profile Card
        Card(
            modifier = Modifier.fillMaxWidth(),
            colors = CardDefaults.cardColors(
                containerColor = MaterialTheme.colorScheme.primaryContainer
            )
        ) {
            Column(modifier = Modifier.padding(16.dp)) {
                Text(
                    text = dashboard.profile.fullName,
                    style = MaterialTheme.typography.headlineSmall
                )
                Text(
                    text = dashboard.profile.rollNumber,
                    style = MaterialTheme.typography.bodyMedium
                )
            }
        }

        // Attendance Stats
        AttendanceCard(stats = dashboard.overallAttendance)

        // Subject Wise
        SubjectsSection(subjects = dashboard.subjectWise)
    }
}

```

4. Location Services

```

@Composable
fun rememberLocationState(): LocationState {
    val context = LocalContext.current

    return remember {
        LocationState(

```

```

        fusedLocationClient = LocationServices.getFusedLocationProviderClient(context)
    )
}

class LocationState(private val fusedLocationClient: FusedLocationProviderClient) {
    fun getLastLocation(callback: (latitude: Double, longitude: Double) -> Unit) {
        // Get last known location
        fusedLocationClient.lastLocation
            .addOnSuccessListener { location ->
                if (location != null) {
                    callback(location.latitude, location.longitude)
                }
            }
    }
}

```

5. ML Kit Face Detection

```

class FaceDetectionManager(context: Context) {
    private val detector = FaceDetection.getClient()

    fun detectFace(bitmap: Bitmap): List<Face> {
        val image = InputImage.fromBitmap(bitmap, 0)
        return detector.process(image)
            .addOnSuccessListener { faces ->
                // Process faces
            }
            .result as List<Face>
    }

    fun generateEmbedding(face: Face): String {
        // Google ML Kit provides face bounding box, landmarks
        // Use custom model or API to generate embedding
        return faceEmbeddingModel.embed(face)
    }
}

```

State Management

ViewModel Architecture

```

View (Composable)
  ↓
ViewModel (StateFlow<UiState>)
  ↓
Repository (suspend functions)
  ↓
ApiService / LocalDB
  ↓
Server / Device Storage

```

Example - Reactive State:

```

@HiltViewModel
class LoginViewModel @Inject constructor(
    private val authRepository: AuthRepository,
    private val sessionManager: SessionManager
) : ViewModel() {

    private val _loginState = MutableStateFlow<UiState<Unit>>(UiState.Idle())

```

```

    val loginState = _loginState.asStateFlow()

    private val _email = MutableStateFlow("")
    val email = _email.asStateFlow()

    fun login(password: String) {
        viewModelScope.launch {
            _loginState.value = UiState.Loading()
            authRepository.login(_email.value, password)
                .onSuccess {
                    _loginState.value = UiState.Success(Unit)
                }
                .onFailure { error ->
                    _loginState.value = UiState.Error(error.message ?: "Login failed")
                }
        }
    }
}

// In Composable
val loginState by viewModel.loginState.collectAsState()
when (val state = loginState) {
    is UiState.Loading -> LoadingDialog()
    is UiState.Success -> LaunchedEffect(Unit) { navigate() }
    is UiState.Error -> ErrorSnackbar(state.message)
}

```

Security & Permissions

Runtime Permissions

Required Permissions (AndroidManifest.xml):

```

<!-- Location -->
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

<!-- Camera (Face Detection) -->
<uses-permission android:name="android.permission.CAMERA" />

<!-- Notifications -->
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />

<!-- Internet -->
<uses-permission android:name="android.permission.INTERNET" />

<!-- Network State -->
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />

```

Permission Request (Runtime):

```

@Composable
fun AttendanceScreen(viewModel: AttendanceViewModel) {
    val locationPermission = rememberPermissionState(
        android.Manifest.permission.ACCESS_FINE_LOCATION
    )
    val cameraPermission = rememberPermissionState(
        android.Manifest.permission.CAMERA
    )

    Column {
        Button(

```

```

        onClick = {
            if (locationPermission.status.isGranted && cameraPermission.status.isGranted) {
                viewModel.markAttendance()
            } else {
                locationPermission.launchPermissionRequest()
                cameraPermission.launchPermissionRequest()
            }
        }
    ) {
        Text("Mark Attendance")
    }
}

```

Data Security

Encryption: - Face embeddings encrypted with AES-256 (backend) - Session tokens in secure SharedPreferences
- SSL/TLS for API communication

Storage: - Encrypted local database with Room + SQLCipher - SessionManager uses EncryptedSharedPreferences

Summary

SAMS Android App Architecture:

1. **Data Layer:** API Service + Repository + Local Storage
2. **Domain Layer:** ViewModels + Business Logic
3. **Presentation Layer:** Jetpack Compose UI + Navigation
4. **Infrastructure:** Location, ML Kit, FCM Services

Key Technologies: - Kotlin + Coroutines for async operations - Jetpack Compose for modern UI - MVVM + Repository pattern - Hilt for dependency injection - ML Kit for face detection - Firebase for notifications

Security: - Runtime permissions handling - Encrypted data storage - SSL/TLS communication - Encrypted face data